William Jackson

Senior Software Engineer - Rust

Wellington, Nevada, United States | +1 (786) 949-7561| williamjdev@outlook.com | linkedin.com/in/william-jackson-40036a3ba

PROFESSIONAL SUMMARY

Senior Software Engineer with **10+ years** of experience building **cloud-native backend platforms**, **event-driven systems**, and **high-reliability data services** across the **networking, operational data, and platform engineering industry**, with deep hands-on expertise in **Rust 1.56–1.75**, **Go 1.17–1.22**, and **Python 3.8–3.12** for performance-sensitive and business-critical workloads. Strong record of improving **p95 latency**, **throughput**, and **production stability** by implementing **idempotency**, **backpressure**, **retry controls**, **structured logging**, **distributed tracing**, and resilient queue-driven workflows. Experienced in designing **secure APIs**, modernizing legacy services, resolving complex production defects, and operating scalable systems in **AWS-first** environments with additional exposure to **Azure** and **GCP**.

SKILLS

- **Languages & Core — Rust 1.56–1.75**, **Go 1.17–1.22**, **Python 3.8–3.12**, TypeScript 4.x–5.x, JavaScript ES6+, SQL, Bash, Linux
- **Backend & APIs —** Rust async services, Tokio, REST APIs, gRPC, Protobuf, middleware/interceptors, rate limiting, timeouts, retries, backpressure, background workers, WebSockets, idempotency patterns, profiling, concurrency tuning
- **Frontend — React 17–18**, **Next.js 12–14**, HTML5, CSS3, Tailwind CSS, Webpack, Vite
- **Data, Messaging & Search —** PostgreSQL, MySQL, SQL Server, Redis, MongoDB, DynamoDB, AWS SQS/SNS, Kafka, RabbitMQ, Elasticsearch, OpenSearch, Redshift, BigQuery
- **Cloud & DevOps —** AWS: EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM | Azure: AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights | GCP: GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery | Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI
- **Observability, Quality & Security —** OpenTelemetry, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch, structured logging, Jest, Mocha, Chai, React Testing Library, Cypress, Playwright, Pact-style contract testing, OAuth2, OpenID Connect, JWT, Okta, Auth0, AWS Cognito, RBAC/ABAC, audit logging, rate limiting

WORK HISTORY

Senior Software Engineer
February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Led development of a high-throughput **Rust** telemetry ingestion platform that processed burst-heavy network events, where **worker pools**, **bounded queues**, and carefully tuned **backpressure** controls reduced ingestion collapse during traffic spikes and stabilized sustained processing under heavy load.
- Reworked a fragile retry flow that had been producing duplicate records during upstream reconnects by implementing **idempotency keys**, replay-safe offset handling, and deduplication windows, which cut double-counting incidents by **90%** and improved trust in downstream analytics.
- Built a unified **gRPC** and **REST** gateway layer that standardized **authentication**, **timeouts**, **request validation**, and error contracts, reducing partner-side integration defects by **35%** and making service behavior far easier to debug across environments.
- Strengthened service security by implementing **OAuth2**, **OIDC**, **JWT validation**, and **RBAC/ABAC** authorization rules across internal and external APIs, closing inconsistent permission gaps that had previously caused policy drift between services.
- Introduced end-to-end **OpenTelemetry** instrumentation with trace correlation, request metadata propagation, and actionable service tags exported into **Datadog**, which reduced mean time to identify latency regressions by **50%** during production investigations.
- Designed resilient event-processing workflows using **Kafka** and **AWS SQS/SNS** with **DLQ routing**, poison-message isolation, and retry guardrails, solving repeated consumer stalls that had been triggered by malformed payloads and upstream schema inconsistencies.

- Built and optimized indexing flows into **Elasticsearch** and **OpenSearch**, resolving a recurring issue where high-cardinality query patterns were overloading transactional databases, and improved search responsiveness for operational users by **40%**.
- Improved **PostgreSQL** performance through targeted indexing, lock-aware batch writes, and query plan analysis while also refining **Redis** caching behavior, which stabilized **p95 latency** by **45%** on the busiest read and ingestion paths.
- Designed mixed persistence patterns across **RDS** and **DynamoDB**, using relational guarantees for strict transactional workloads and key-based NoSQL access for scale-sensitive paths, eliminating a class of contention issues caused by forcing all workloads into one storage model.
- Owned production rollout strategy on **EKS** with canary analysis and staged traffic promotion while keeping selected support services on **ECS**, which lowered rollout risk and reduced rollback frequency by **30%** after major backend changes.
- Implemented orchestration for long-running workflows using **EventBridge** and **Step Functions**, replacing brittle script-based recovery processes and improving retry semantics, visibility, and operator confidence during partial-failure scenarios.
- Standardized **structured logging**, **audit trails**, **CloudWatch** metrics, **Prometheus/Grafana** dashboards, and tighter **IAM** plus secrets handling, then connected everything to CI/CD through **GitHub Actions**, **GitLab CI**, and **Jenkins**, producing faster rollback paths and more reliable releases.

Software Engineer
October 2018 to January 2021

Avantia, Inc. - Ohio, United States

- Built high-performance **Rust** utilities for feature generation pipelines, optimizing parsing logic, memory allocation patterns, and file I/O behavior so nightly compute jobs completed faster and reduced tail-latency pressure on downstream scoring workloads.
- Developed **Python** ETL services that normalized inconsistent operational feeds into training-ready datasets, resolving repeated timestamp drift and identifier mismatch issues that had previously caused corrupted joins and unreliable historical comparisons.
- Redesigned several **PostgreSQL** feature tables with better partition-aware indexing, query tuning, and materialized extraction outputs, improving SLA adherence and reducing heavy join execution time by **38%** for nightly production runs.
- Implemented expectation-style validation and quarantine flows that isolated malformed source records before they contaminated downstream pipelines, preventing silent data quality failures and significantly improving confidence in model input integrity.
- Added incremental backfill tooling with watermark checkpointing and duplicate-safe replay behavior, solving a long-standing issue where historical reprocessing overloaded source systems and occasionally introduced duplicate rows into feature tables.
- Built internal **REST** scoring APIs with explicit error contracts, **timeouts**, **retry rules**, and circuit-breaker safeguards, reducing cascading failures when dependent services slowed down and improving client-side reliability under partial outages.
- Introduced correlated logging and metric instrumentation across jobs and APIs so a single customer record could be traced through ingestion, transformation, and scoring, which made operational investigations much faster during incident response.
- Improved runtime cost and processing stability by optimizing serialization formats, reducing wasteful object transformations, and caching repeated lookup patterns in **Redis**, which improved repeated query efficiency by roughly **25%**.
- Integrated asynchronous task handling with **RabbitMQ** and **AWS SQS/SNS**, including proper dead-letter routing and retry boundaries, resolving poison-message loops that had been causing repeated job starvation in shared worker pools.
- Delivered operational data exploration using **Elasticsearch** and supported downstream exports into **Redshift** and **BigQuery**, giving analytics teams faster access to trustworthy data without pushing more ad hoc traffic onto core transactional stores.
- Containerized services with **Docker**, deployed workloads into **Kubernetes** including **AKS**, and standardized build and release automation through **Azure DevOps** and **CircleCI**, improving release consistency and making rollback execution much safer.

Software Developer March 2016 to September 2018
ArnAmy, Inc. - Florida, United States

- Delivered backend features using **Python** services for operational data workflows and introduced selective **Rust** components for CPU-heavy transformation paths, reducing batch execution time and improving runtime stability for larger workloads.
- Built **REST APIs** that exposed normalized datasets with well-defined contracts, pagination, and validation behavior, fixing earlier client issues where oversized responses and inconsistent schemas caused repeated integration failures.
- Optimized **MySQL** and **PostgreSQL** workloads through index design, join rewrites, and query profiling, which shortened batch windows and improved report freshness for stakeholders who depended on near-daily operational visibility.
- Implemented background workers with retry safety and **idempotency** protections, eliminating manual rerun patterns that had previously been required whenever transient network problems interrupted scheduled processing jobs.
- Added **Redis** caching with standardized cache-key strategy and invalidation rules, improving stability for read-heavy endpoints and preventing repeated database pressure during peak internal reporting usage.
- Introduced **Elasticsearch** indexing for internal discovery workflows, solving a recurring usability issue where teams had been relying on slow database filters to find records across growing operational datasets.
- Built reconciliation utilities that compared source counts, transformed outputs, and final delivery totals, helping detect drift early and preventing long-running discrepancies that were difficult to unwind after multiple batch cycles.
- Adopted **Docker**-based development environments to remove machine-specific dependency issues, which reduced onboarding friction and minimized “works on my machine” defects during collaborative feature delivery.
- Created lightweight **Linux** runbooks, health checks, and operational recovery steps that improved visibility into service failure modes and made recurring incidents easier to resolve without escalations.
- Added **structured logging**, baseline **CloudWatch** telemetry, and simple **Grafana** dashboards, which gave the team earlier warning signs during production issues and reduced time spent diagnosing vague service behavior.

Junior Software Developer October 2014 to February 2016
Centric Tech Inc. - New Jersey, United States

- Wrote **Python** and **SQL** scripts to clean inconsistent customer and reporting datasets, adding validation rules that stopped export failures caused by format drift, missing fields, and duplicate source records.
- Built small **Bash** CLI tools for nightly pulls, file comparisons, and audit generation, reducing manual spreadsheet work and creating more reliable repeatable outputs for internal reporting operations.
- Implemented statistical summaries and anomaly flags for business metrics, helping non-technical stakeholders identify unusual movements earlier instead of discovering data-quality problems after reports were already shared.
- Improved data extraction reliability by standardizing input checks, error messaging, and output formatting, which reduced repeated support requests caused by inconsistent batch results across teams.
- Strengthened scheduled job behavior by introducing clearer exit codes and structured error summaries, making failure triage faster and reducing guesswork when overnight processes did not complete successfully.
- Partnered with senior engineers to document source ownership, field definitions, and transformation logic, helping remove duplicated effort and reducing confusion around conflicting business terminology.
- Optimized a slow reporting query through schema cleanup, index redesign, and simpler access patterns, bringing runtime down enough to support more iterative analysis and stakeholder follow-up requests.
- Supported incremental delivery by breaking larger requests into smaller low-risk changesets, which improved deployment safety and lowered regression risk when multiple reporting updates were moving at once.
- Contributed to troubleshooting on **Linux** hosts using practical runbooks and repeatable recovery steps, helping the team respond faster to recurring operational issues without reinventing the diagnosis process.

EDUCATION

Bachelor's degree in Computer Science 2010 to 2014
Stanford University Department of Computer Science

- Focused on strong fundamentals in **software engineering**, **databases**, and **systems**, building a foundation that translates into production-first delivery and disciplined operational practices.

CERTIFICATIONS

- **AWS Certified Developer – Associate**
  Validated hands-on capability in building, deploying, troubleshooting, and securing production application workloads across core **AWS** services using modern development and integration practices.
- **HashiCorp Certified: Terraform Associate**
  Demonstrated practical knowledge of **Infrastructure as Code**, reusable provisioning patterns, state management, and safe infrastructure automation across cloud-native delivery environments.
- **Certified Kubernetes Application Developer (CKAD)**
  Confirmed applied skill in designing, deploying, troubleshooting, and operating containerized applications on **Kubernetes** with strong focus on reliability, configuration, scaling, and rollout safety.